

BE TechSpec

📋 문서 메타

항목	내용
문서	BE TechSpec — gohae v1.0
상위 문서	기획문서
버전	v1.2 (리뷰 라운드 2 반영)
최종 업데이트	@2026년 5월 16일
작성자	오준경 (jjuney)
대상 독자	본인 (1인 개발) — 12시간 스프린트 중 구현 가이드



스코프 — Next.js Route Handlers/Server Actions + Supabase Postgres(pgvector) + Anthropic/OpenAI 통합. 화면·UI 레이어는 [FE TechSpec](#) 참조.



v1.1 → v1.2 변경 요지 — (1) persona 존재 체크를 메시지 INSERT **이전으로 이동** (고아 데이터 방지). (2) 활성 conversation lookup에 `isNull(endedAt)` 추가 (종료된 대화에 메시지 INSERT 방지). (3) `InputSchema.role` 을 `['user', 'assistant']` 로 좁힘 (system role 인젝션 차단). (4) FE v1.2 정합: 위기 신호 인터페이스 표를 `initialCrisisIds · liveCrisis` 변수명으로 갱신 + `x-conversation-id` 응답 헤더 명세 추가. (5) Retrieval 시간 가중치 30 → **60일** + 검증 슬롯 박음. (6) Daily cap 50의 **비용 근거** 결정 로그에 추가. (7) 메모리 추출 max **20** → **8** + 프롬프트에 "가장 중요한 3-5개" 지시. (8) **Vercel Pro 플랜 가정** 명시 (Hobby 조정 가이드 포함). (9) `onFinish` Vercel disconnect 보장 한계 명시. (10) `placeholder` 영구 pending sweep v1.x. (11) Idempotency-key v1.x. (12) Prompt caching v1.x. (13) 데이터 삭제 SLA **요청 채널 명시**. (14) 보안 체크리스트에 **CSRF 의식적 결정** 추가.

1. 기술 스택

영역	선택	비고
Server	Next.js 15 Route Handlers + Server Actions	Django 제외
Hosting	Vercel (Pro 플랜 가정)	maxDuration 60s 사용. Hobby는 \$11 참조
DB	Supabase Postgres + pgvector (hnsw)	region: ap-northeast-2
ORM	Drizzle ORM (introspect 단방향)	SQL이 단일 진실 공급원
Auth	Supabase Auth (Google OAuth)	<code>@supabase/ssr</code>
LLM	Anthropic Claude Sonnet 4.6	대화 + 메모리 추출
임베딩	OpenAI <code>text-embedding-3-small</code> (1536d)	Anthropic 임베딩 API 없음
Validation	zod	API 입력 검증 + 인젝션 차단
모니터링	@sentry/nextjs	scrubbing 필수
Runtime	<code>runtime = 'nodejs'</code> (Drizzle 쓰는 모든 핸들러)	postgres-js Edge 미동작

2. 폴더 구조 (BE 관련)

```
supabase/migrations/
├─ 0001_init.sql          # 단일 진실 공급원
└─ 0002_rls.sql

lib/
├─ db/{index,schema}.ts   # drizzle (introspect 단방향)
├─ supabase/{client,server,middleware}.ts
├─ ai/
│   ├─ claude.ts
│   ├─ embeddings.ts
│   ├─ prompts/{system,memory-extract}.ts
│   └─ memory/{extract,retrieve}.ts
├─ crisis/keywords.ts
└─ auth.ts

app/api/
├─ chat/route.ts          # POST 스트리밍
├─ conversation/end/route.ts # POST (외부/모바일 호출용 - v1.0 FE는 미사
응)
└─ health/route.ts
```

3. 데이터 모델

3.1 SQL — 단일 진실 공급원

```
-- 0001_init.sql

create extension if not exists vector;

create table personas (
  user_id      uuid primary key references auth.users(id) on delete cascade,
  nickname     text not null,
  age_range    text not null check (age_range in ('10s','20s','30s','40s'+')),
  tone         text not null default 'formal' check (tone in ('formal','casual')),
  self_intro   text,
  created_at   timestampz not null default now(),
  updated_at   timestampz not null default now()
);

create table conversations (
  id           uuid primary key default gen_random_uuid(),
```

```

    user_id          uuid not null references auth.users(id) on delete cascade,
    started_at       timestampz not null default now(),
    ended_at         timestampz,
    summary_status    text not null default 'pending'
        check (summary_status in ('pending', 'completed', 'failed'))
);
create index conversations_user_active_idx
on conversations(user_id, ended_at);

-- 한 사용자당 활성 대화 1개 (단순화)
create unique index conversations_one_active_per_user
on conversations(user_id)
where ended_at is null;

create table messages (
    id                bigserial primary key,
    conversation_id    uuid not null references conversations(id) on delete cascade,
    role              text not null check (role in ('user', 'assistant', 'system')),
    -- △ v1.0 코드는 system을 INSERT하지 않음. SQL은 보수적으로 좁히고 싶지만
    -- 마이그레이션 비용 회피 위해 v1.0은 그대로 두고 zod에서 차단($6.1).
    -- 결정 로그·v1.x 미해결 참조.
    content            text not null default '',
    status             text not null default 'completed'
        check (status in ('pending', 'completed', 'failed')),
    crisis_flag        boolean not null default false,
    created_at         timestampz not null default now(),
    completed_at       timestampz
);
create index messages_conv_time_idx on messages(conversation_id, created_at);

create table memories (
    id                uuid primary key default gen_random_uuid(),
    user_id           uuid not null references auth.users(id) on delete cascade,
    type              text not null check (type in (
        'relationship', 'recurring_issue', 'emotional_pattern',
        'trigger', 'preference', 'fact'
    )),
    content            text not null,
    embedding          vector(1536),
    status            text not null default 'active' check (status in ('active', 'superseded')),
    created_at        timestampz not null default now()
);
create index memories_user_status_idx on memories(user_id, status);

```

```
create index memories_embedding_hnsw_idx
on memories using hnsw (embedding vector_cosine_ops);
```

3.2 Drizzle 스키마 (introspect 결과)

 SQL이 진실. `drizzle-kit introspect` 로 자동 생성, 손편집 금지. 양방향 수동 동기화 금지.

```
// lib/db/schema.ts
import { pgTable, uuid, text, timestamp, boolean, bigserial, index, custom
Type } from 'drizzle-orm/pg-core'

const vector = customType<{ data: number[]; driverData: string }>({
  dataType() { return 'vector(1536)' },
  toDriver(value) { return `[${value.join(',')}]` },
  fromDriver(value) { return JSON.parse(value) },
})

export const personas = pgTable('personas', {
  userId: uuid('user_id').primaryKey(),
  nickname: text('nickname').notNull(),
  ageRange: text('age_range').notNull().$type<'10s'|'20s'|'30s'|'40s+'>(),
  tone: text('tone').notNull().default('formal').$type<'formal'|'casual'>
  (),
  selfIntro: text('self_intro'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().def
aultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).notNull().def
aultNow(),
})

export const conversations = pgTable('conversations', {
  id: uuid('id').primaryKey().defaultRandom(),
  userId: uuid('user_id').notNull(),
  startedAt: timestamp('started_at', { withTimezone: true }).notNull().def
aultNow(),
  endedAt: timestamp('ended_at', { withTimezone: true }),
  summaryStatus: text('summary_status').notNull().default('pending')
  .$type<'pending'|'completed'|'failed'>(),
}, (t) => ({
  userActiveIdx: index('conversations_user_active_idx').on(t.userId, t.end
edAt),
}))

export const messages = pgTable('messages', {
  id: bigserial('id', { mode: 'number' }).primaryKey(),
```

```

conversationId: uuid('conversation_id').notNull()
  .references(() => conversations.id, { onDelete: 'cascade' }),
role: text('role').notNull().$type<'user'|'assistant'|'system'>(),
content: text('content').notNull().default(''),
status: text('status').notNull().default('completed')
  .$type<'pending'|'completed'|'failed'>(),
crisisFlag: boolean('crisis_flag').notNull().default(false),
createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
completedAt: timestamp('completed_at', { withTimezone: true }),
}, (t) => ({
  convTimeIdx: index('messages_conv_time_idx').on(t.conversationId, t.createdAt),
}))

export const memories = pgTable('memories', {
  id: uuid('id').primaryKey().defaultRandom(),
  userId: uuid('user_id').notNull(),
  type: text('type').notNull().$type<
    'relationship'|'recurring_issue'|'emotional_pattern'|'trigger'|'preference'|'fact'
  >(),
  content: text('content').notNull(),
  embedding: vector('embedding'),
  status: text('status').notNull().default('active').$type<'active'|'superseded'>(),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
}, (t) => ({
  userStatusIdx: index('memories_user_status_idx').on(t.userId, t.status),
}))

```

3.3 Drizzle 인스턴스

```

// lib/db/index.ts
import { drizzle } from 'drizzle-orm/postgres-js'
import postgres from 'postgres'
import * as schema from './schema'

const client = postgres(process.env.DATABASE_URL!, { prepare: false })
export const db = drizzle(client, { schema })

```

4. RLS — 정직한 운영 기준



v1.0 Drizzle 경로에서 RLS는 미적용. Supabase의 `DATABASE_URL` 은 슈퍼유저 권한 → RLS bypass.
모든 쿼리에 `user_id` 필터 의무.

4.1 RLS Policy (baseline)

```
-- 0002_rls.sql
alter table personas enable row level security;
alter table conversations enable row level security;
alter table messages enable row level security;
alter table memories enable row level security;

create policy personas_self on personas
  for all using (user_id = auth.uid()) with check (user_id = auth.uid());

create policy conversations_self on conversations
  for all using (user_id = auth.uid()) with check (user_id = auth.uid());

create policy messages_self on messages
  for all using (
    exists (select 1 from conversations c
      where c.id = messages.conversation_id and c.user_id = auth.uid())
  ) with check (
    exists (select 1 from conversations c
      where c.id = messages.conversation_id and c.user_id = auth.uid())
  );

create policy memories_self on memories
  for all using (user_id = auth.uid()) with check (user_id = auth.uid());
```

4.2 RLS의 실제 가치 정리

시나리오	RLS 적용
Drizzle/postgres 직접 (v1.0 정상 경로)	❌ bypass
향후 <code>supabase-js</code> 경로	✅ 적용
Supabase Studio "as user" 모드	✅ 적용
<code>service_role_key</code>	❌ bypass

4.3 셀프 PR 리뷰 체크리스트

- ☐ DB 쿼리 `where` 절에 `user_id = currentUser.id` 검증?
- ☐ 사용자 입력 ID는 `where (id = ? and user_id = currentUser.id)` 패턴?
- ☐ Route Handler/Server Action 진입부 `getCurrentUser()` 게이트?
- ☐ **persona** 의존 흐름은 메시지 **INSERT** 전에 **persona** 체크? (§6.1 사고 방지)

5. 인증 헬퍼

```
// lib/auth.ts
import { cache } from 'react'
import { eq } from 'drizzle-orm'
import { createClient } from './supabase/server'
import { db } from './db'
import { personas } from './db/schema'

export const getCurrentUser = cache(async () => {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  return user
})

export const getCurrentPersona = cache(async () => {
  const user = await getCurrentUser()
  if (!user) return null
  return (await db.query.personas.findFirst({
    where: eq(personas.userId, user.id)
  }))? null
})
```

6. API 엔드포인트

6.0 FE ↔ BE 인터페이스 (단일 진실 표, v1.2)

채널	형태	BE 세팅	FE 소비	용도
응답 헤더 <code>x-crisis: 1</code>	string	<code>/api/chat</code> 위기 키워드 감지 시	<code>useChat.onResponse(res)</code> → <code>setLiveCrisis(true)</code>	활성 stream 위기 신호 (a11y alert)
응답 헤더 <code>x-conversation-id</code>	uuid	항상 (<code>/api/chat</code> 응답 예)	<code>useChat.onResponse(res)</code> → <code>setActiveId(newId)</code>	새 conversation 생성 시 클라이언트 동기화
DB 컬럼 <code>messages.crisis_flag</code>	boolean	<code>/api/chat</code> user 메시지 INSERT 시	<code>chat/page.tsx</code> RSC가 history에서 추출 → <code>initialCrisisIds: string[]</code> prop → 클라이언트 <code>crisisIds: Set<string></code>	새로고침·재방문 시 카드 복원
DB 컬럼 <code>messages.status</code>	enum	INSERT/UPDATE 시	(v1.0 미사용 — v1.x에서 placeholder UI 분기)	외로운 user 메시지 방지, 실패 마킹
시스템 프롬프트 <code>crisis_section</code>	system instruction	<code>buildSystemPrompt({ crisis: true })</code>	Claude (LLM)	위기 상황 인지·신중 응대

모든 채널은 메시지 콘텐츠와 분리. **sentinel 문자열 사용 금지.**

6.1 POST /api/chat — placeholder 패턴 + 인터페이스 정합

책임 순서 (★ 표시는 v1.2 변경):

1. 사용자 인증
2. ★ persona 존재 확인 (메시지 INSERT 전에)
3. 일일 cap 확인 (KST 기준)
4. ★ 활성 conversation lookup에 `isNull(endedAt)` 적용 — 종료된 ID 재사용 차단 → 새 대화 생성으로 fallback
5. user 메시지 INSERT (status='completed', crisis_flag 계산)
6. assistant placeholder INSERT (status='pending', content='')
7. Retrieval + 시스템 프롬프트 조립
8. Claude 스트리밍
9. `onFinish`: placeholder UPDATE → completed
10. `onError` / `onAbort` / timeout: placeholder UPDATE → failed
11. ★ 응답 헤더 `x-conversation-id: <conv.id>` 항상 세팅 + 위기 시 `x-crisis: 1`

코드

```
// app/api/chat/route.ts
import { NextResponse } from 'next/server'
import { streamText } from 'ai'
import { anthropic } from '@ai-sdk/anthropic'
import { z } from 'zod'
import * as Sentry from '@sentry/nextjs'
import { sql, and, eq, isNull, desc } from 'drizzle-orm'
import { db } from '@/lib/db'
import { conversations, messages, personas } from '@/lib/db/schema'
import { getCurrentUser } from '@/lib/auth'
import { containsCrisisSignal } from '@/lib/crisis/keywords'
import { retrieveMemories } from '@/lib/ai/memory/retrieve'
import { buildSystemPrompt } from '@/lib/ai/prompts/system'

export const runtime = 'nodejs'
export const maxDuration = 60 // Vercel Pro 가정. Hobby 운영 시 $11 참조

// ★ system role 차단 - 프롬프트 인젝션 방어
const InputSchema = z.object({
  messages: z.array(z.object({
    role: z.enum(['user', 'assistant']),
    content: z.string().max(8000),
  })),
  conversationId: z.string().uuid().nullable(),
})
```

```

export async function POST(req: Request) {
  const user = await getCurrentUser()
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })

  const parsed = InputSchema.safeParse(await req.json())
  if (!parsed.success) return NextResponse.json({ error: 'Invalid input' }, { status: 400 })

  const { messages: input, conversationId } = parsed.data
  const lastUser = input.filter(m => m.role === 'user').at(-1)
  if (!lastUser) return NextResponse.json({ error: 'No user message' }, { status: 400 })

  // ★ 2. persona 체크를 INSERT 전에
  const persona = await db.query.personas.findFirst({
    where: eq(personas.userId, user.id)
  })
  if (!persona) return NextResponse.json({ error: 'Persona required' }, { status: 400 })

  // 3. 일일 cap (KST)
  const [{ count }] = await db.execute<{ count: number }>(sql`
    select count(*)::int as count
    from messages m
    join conversations c on c.id = m.conversation_id
    where c.user_id = ${user.id}::uuid
    and m.role = 'user'
    and m.created_at >= (now() at time zone 'Asia/Seoul')::date at time
zone 'Asia/Seoul'
`)
  if (count >= 50) {
    return NextResponse.json(
      { error: '오늘은 충분히 이야기했어요. 내일 다시 만나요.' },
      { status: 429 }
    )
  }

  // ★ 4. 활성 conversation 확보 - isNull(endedAt) 필터
  let conv = conversationId
  ? await db.query.conversations.findFirst({
    where: and(
      eq(conversations.id, conversationId),
      eq(conversations.userId, user.id),
      isNull(conversations.endedAt), // ★ 종료된 대화 재사용 차단
    )
  })
  : null

```

```

if (!conv) {
  const [created] = await db.insert(conversations)
    .values({ userId: user.id })
    .returning()
  conv = created
}

// 5. user 메시지 + 위기 감지
const crisis = containsCrisisSignal(lastUser.content)
await db.insert(messages).values({
  conversationId: conv.id,
  role: 'user',
  content: lastUser.content,
  status: 'completed',
  crisisFlag: crisis,
})

// 6. assistant placeholder
const [placeholder] = await db.insert(messages).values({
  conversationId: conv.id,
  role: 'assistant',
  content: '',
  status: 'pending',
}).returning({ id: messages.id })

// 7. Retrieval (마지막 user 메시지 3개 합쳐)
const recentUserMsgs = input.filter(m => m.role === 'user').slice(-3).map(m => m.content)
const retrieved = await retrieveMemories(user.id, recentUserMsgs, 5)
const system = buildSystemPrompt({ persona, memories: retrieved, crisis
})

// 8-10. Stream
const result = await streamText({
  model: anthropic('claude-sonnet-4-6'),
  system,
  messages: input,
  abortSignal: AbortSignal.timeout(45_000),
  onFinish: async ({ text }) => {
    try {
      await db.update(messages)
        .set({ content: text, status: 'completed', completedAt: new Date
() })
        .where(eq(messages.id, placeholder.id))
    } catch (e) { Sentry.captureException(e) }
  },
  onError: async ({ error }) => {
    Sentry.captureException(error)
  }
})

```

```

    try {
      await db.update(messages)
        .set({ content: '응답을 가져오지 못했어요.', status: 'failed', completedAt: new Date() })
        .where(eq(messages.id, placeholder.id))
    } catch (e) { Sentry.captureException(e) }
  },
})

// 11. 응답 헤더 - 항상 x-conversation-id, 위기 시 x-crisis
const res = result.toDataStreamResponse()
res.headers.set('x-conversation-id', conv.id) // ★ 항상 세팅
if (crisis) res.headers.set('x-crisis', '1')
return res
}

```

onFinish 보장 한계 & idempotency 약점 — 의식적 결정

항목	현실	v1.0 대응	v1.x
Vercel disconnect 시 onFinish 실행	Serverless는 응답 close 후 비동기 보장 안 됨. AI SDK가 stream 종료 직전 호출하도록 설계되어 보통은 OK이지만, 중간 disconnect 시 partial/abort로 호출되거나 누락 가능	placeholder가 영구 pending 으로 남을 수 있음. Sentry로 가시화	cron sweep: pending 상태 5분 이상 → failed 로 마킹
중복 호출 (네트워크 재시도/F5/더블클릭)	user/assistant 메시지 쌍이 또 INSERT 됨. FE의 입력 disable만이 방어	결정 로그에 명시. v1.0 허용.	idempotency-key 헤더 도입 (Idempotency-Key: <client-uuid>) → 동일 키 1분 내 재호출은 같은 응답 반환

6.2 POST /api/conversation/end

i v1.0 FE는 이 엔드포인트를 호출하지 않는다. Server Action에서 **extractAndStoreMemories** 를 직접 호출(FE TechSpec §8.4). 이 route는 외부/모바일 호출용으로 유지.

```

// app/api/conversation/end/route.ts
import { NextResponse } from 'next/server'
import { z } from 'zod'
import * as Sentry from '@sentry/nextjs'
import { and, eq, isNull, asc } from 'drizzle-orm'
import { db } from '@/lib/db'
import { conversations, messages } from '@/lib/db/schema'
import { getCurrentUser } from '@/lib/auth'
import { extractAndStoreMemories } from '@/lib/ai/memory/extract'

export const runtime = 'nodejs'
export const maxDuration = 60

```

```

const Input = z.object({ conversationId: z.string().uuid() })

export async function POST(req: Request) {
  const user = await getCurrentUser()
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status:
401 })

  const { success, data } = Input.safeParse(await req.json())
  if (!success) return NextResponse.json({ error: 'Invalid' }, { status: 4
00 })

  // 원자적 UPDATE+RETURNING - race 방지
  const claimed = await db.update(conversations)
    .set({ endedAt: new Date() })
    .where(and(
      eq(conversations.id, data.conversationId),
      eq(conversations.userId, user.id),
      isNull(conversations.endedAt),
    ))
    .returning({ id: conversations.id })

  if (claimed.length === 0) {
    return NextResponse.json({ ok: true, alreadyEnded: true })
  }

  const msgs = await db.query.messages.findMany({
    where: eq(messages.conversationId, data.conversationId),
    orderBy: [asc(messages.createdAt)]
  })

  try {
    await extractAndStoreMemories(user.id, msgs)
    await db.update(conversations).set({ summaryStatus: 'completed' })
      .where(eq(conversations.id, data.conversationId))
    return NextResponse.json({ ok: true, extracted: true })
  } catch (e) {
    Sentry.captureException(e)
    await db.update(conversations).set({ summaryStatus: 'failed' })
      .where(eq(conversations.id, data.conversationId))
    return NextResponse.json({ ok: true, extracted: false })
  }
}

```

6.3 GET /api/health

```
// app/api/health/route.ts
import { NextResponse } from 'next/server'
import { sql } from 'drizzle-orm'
import { db } from '@/lib/db'

export const runtime = 'nodejs'

export async function GET() {
  try {
    await db.execute(sql`select 1`)
    return NextResponse.json({ ok: true, db: 'up', ts: Date.now() })
  } catch (e) {
    return NextResponse.json({ ok: false, db: 'down' }, { status: 503 })
  }
}
```

| 외부 모니터링은 v1.x.

7. Claude 통합 — 시스템 프롬프트

7.1 buildSystemPrompt

```
// lib/ai/prompts/system.ts
type Persona = { nickname: string; ageRange: '10s'|'20s'|'30s'|'40s+'; tone: 'formal'|'casual'; selfIntro: string | null }

export function buildSystemPrompt(args: {
  persona: Persona
  memories: { type: string; content: string; createdAt: Date }[]
  crisis: boolean
}): string {
  const { persona, memories, crisis } = args

  const tone = persona.tone === 'formal'
    ? '존댓말로, 차분하고 정중한 톤을 유지하세요.'
    : '편한 반말로, 친근하지만 거칠지 않게 대화하세요.'

  const persona_section = `## 사용자 정보
- 닉네임: ${persona.nickname}
- 연령대: ${persona.ageRange}
${persona.selfIntro ? `- 자기소개: ${persona.selfIntro}` : ''}`

  const sorted = [...memories].sort((a, b) => b.createdAt.getTime() - a.createdAt.getTime())
  const memory_section = sorted.length === 0
```

```

? '## 사용자 기억\n(아직 누적된 정보 없음)'
: `## 사용자 기억 (이전 대화에서 누적된 사실, 최신순)
${sorted.map(m => ` - [${m.type}] ${m.content}`)}.join('\n')
}

> 동일 주제에 대해 여러 사실이 보이면, 가장 최근 정보를 우선 신뢰하세요.`

const crisis_section = crisis ? `
## △ 위기 상황 인지
사용자의 마지막 메시지에서 자살·자해·극단적 선택과 관련된 신호가 감지되었습니다.
- 사용자를 비판하거나 평가하지 말고, 지금의 마음을 인정·공감하는 것을 우선하세요.
- 즉답으로 해결책을 제시하지 말고, 사용자가 안전하게 머무를 수 있는 대화를 만드세요.
- 1393(자살예방상담전화)에 대한 안내는 별도 카드로 노출되었습니다. 응답에서 **1회만*
*, 자연스럽게 부드럽게 재언급 가능. 반복 강요 금지.
- 의료가기가 아니며 진단·치료를 제공하지 않습니다.
` : ''

return `당신은 "gohae"라는 AI 상담 서비스입니다.
사용자의 일상적 고민·정서적 어려움을 들어주고, 맥락을 기억하며 이어가는 친근한 상대입니
다.

# 응답 원칙
- 마크다운 사용 금지. 일반 텍스트로 답하세요.
- 한 번에 너무 길게 말하지 마세요. 2-4문장이 기본.
- 진단·치료·약물 처방을 하지 않습니다.
- ${tone}
- 사용자 기억을 자연스럽게 활용하되, "기억하고 있어요"를 매번 명시하지 마세요.

${persona_section}

${memory_section}
${crisis_section}`
}

```

7.2 검증 슬롯 (T+11:00~11:30)

본인 첫 사용 시 점검:

1. 위기 안내 어조 — 1393이 매번 강요되지 않는지.
2. 시간 가중치 튜닝 — 한 달 이상 지난 사실이 retrieval에 포함되는지 (§8.3 참조). 차별점 검증.

8. 장기기억 시스템

8.1 메모리 추출 — **max(8)** + "중요한 3-5개" 지시

```

// lib/ai/memory/extract.ts
import { generateObject } from 'ai'
import { z } from 'zod'

```

```
import { anthropic } from '@ai-sdk/anthropic'
import { db } from '@/lib/db'
import { memories } from '@/lib/db/schema'
import { embed } from '../embeddings'

const MemoryItem = z.object({
  type: z.enum(['relationship', 'recurring_issue', 'emotional_pattern', 'trigger', 'preference', 'fact']),
  content: z.string().min(5).max(200),
})
const ExtractSchema = z.object({ items: z.array(MemoryItem).max(8) }) //
★ 20→8
```

```
export async function extractAndStoreMemories(
  userId: string,
  msgs: { role: string; content: string }[]
) {
  const transcript = msgs
    .filter(m => m.role === 'user' || m.role === 'assistant')
    .map(m => `${m.role === 'user' ? '사용자' : '상답자'}: ${m.content}`)
    .join('\n')
    .slice(-8000)

  const { object } = await generateObject({
    model: anthropic('claude-sonnet-4-6'),
    schema: ExtractSchema,
    abortSignal: AbortSignal.timeout(30_000),
    prompt: `다음은 사용자와 AI 상답자의 대화 일부입니다. 사용자에게 대한 **사실** (인물관계, 반복 이슈, 감정 패턴, 트리거, 선호, 기타 사실)을 1-2문장으로 추출하세요.`
  })
```

규칙:

- 사용자 본인의 사실만. 일반론·추측 금지.
- **이 대화에서 가장 중요한 3-5개만.** 무리하게 많이 뽑지 마세요. 메모리 인플레이션은 retrieval 정밀도를 떨어뜨립니다.
- 동일 사실 중복 금지.
- 너무 일시적인 감정은 제외. 패턴/반복성이 있는 것만.
- 모순되는 변화가 있다면 **가장 최근 상태**를 사실로.
- 한국어로 명확하게.

대화:

```
${transcript}`,
  })

  if (object.items.length === 0) return

  const embeddings = await embed(object.items.map(i => i.content))

  await db.insert(memories).values(
```

```

    object.items.map((item, i) => ({
      userId,
      type: item.type,
      content: item.content,
      embedding: embeddings[i],
      status: 'active' as const,
    })))
  )
}

```

8.2 임베딩

```

// lib/ai/embeddings.ts
import OpenAI from 'openai'
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY })

export async function embed(texts: string[]): Promise<number[][]> {
  if (texts.length === 0) return []
  const res = await openai.embeddings.create({ model: 'text-embedding-3-small', input: texts })
  return res.data.map(d => d.embedding)
}

```

8.3 Retrieval — 시간 가중치 60일로 완화



v1.1 30일 → v1.2 60일 변경 사유 — 차별점이 "장기기억"인데 30일 가중치는 짧음. similarity 0.5·30일 후 → 0.18(컷 0.3 미달, 사라짐). 60일로 늘리면 같은 조건에서 0.31(보존). 차별점 보호.

```

// lib/ai/memory/retrieve.ts
import { sql } from 'drizzle-orm'
import { db } from '@lib/db'
import { embed } from '../embeddings'

const TIME_DECAY_DAYS = 60 // ★ v1.1 30→60. T+11:00 검증 슬롯에서 추가 튜닝 (§7.2)
const SCORE_CUTOFF = 0.3

export async function retrieveMemories(
  userId: string,
  recentUserTexts: string[],
  k: number = 5
) {
  const joined = recentUserTexts.filter(t => t.trim().length > 0).join('\n')
  if (joined.trim().length < 10) return []

```

```

const [queryEmb] = await embed([joined])
const vec = `[${queryEmb.join(',')}]`

const rows = await db.execute<{
  type: string; content: string; createdAt: Date; score: number
}>(sql`
  select
    type,
    content,
    created_at as "createdAt",
    (1 - (embedding <=> ${vec}::vector))
    * exp( - extract(epoch from (now() - created_at)) / (${TIME_DECAY_DAYS}.0 * 86400) ) as score
  from memories
  where user_id = ${userId}::uuid
    and status = 'active'
    and embedding is not null
  order by score desc
  limit ${k}
`)

return rows.filter(r => r.score > SCORE_CUTOFF)
}

```

9. 위기 키워드 디텍션

FE TechSpec §9와 동일 모듈 import.

```

// lib/crisis/keywords.ts
export const CRISIS_KEYWORDS = [
  '자살', '자해', '죽고 싶', '사라지고 싶', '없어지고 싶',
  '목매', '뛰어내리', '약을 먹고', '극단적 선택',
  '끝내고 싶', '사라질래', '그만하고 싶',
] as const

export function containsCrisisSignal(text: string): boolean {
  const normalized = text.toLowerCase()
  return CRISIS_KEYWORDS.some(k => normalized.includes(k))
}

```

false negative 한계 인정. LLM 2nd-pass는 v1.x.

10. 외부 API 신뢰성

항목	정책
Timeout	Claude 스트리밍 45s, generateObject 30s, OpenAI embedding 15s. <code>AbortSignal.timeout()</code>
Retry	1회만. LLM 품질 이슈는 retry 의미 없음
Idempotency — chat	placeholder 패턴. 중복 호출 시 두 쌍 INSERT 가능 → v1.0은 FE disable 의존, v1.x에 <code>Idempotency-Key</code> 헤더 도입
Idempotency — conversation/end	원자적 UPDATE+RETURNING으로 먹등
Cost cap	사용자당 일일 user 메시지 50개. KST 자정 기준
Error UX	외부 API 실패 시 placeholder를 <code>failed</code> 로 마킹 + 토스트 + Sentry

Daily cap 50의 비용 근거

Claude Sonnet 4.6 (\$3/M input, \$15/M output) 가정:

항목	값
시스템 프롬프트 평균	~1,500 tokens × 50회 = 75K input → \$0.225
사용자 메시지 평균	~100 tokens × 50회 = 5K input → \$0.015
Assistant 응답 평균	~200 tokens × 50회 = 10K output → \$0.150
OpenAI embedding (retrieval)	~50 tokens × 50회 = 2.5K → \$0.0001
사용자당 일일 합계	~\$0.40
사용자 10명 × 30일	~\$120/월

| v1.0 검증 단계(본인 + 지인 ~5명)엔 안전한 상한. 사용자 늘면 cap 조정 검토.

11. 환경 변수 & Vercel 플랜 가정

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=... # 마이그레이션 전용

# Postgres (Drizzle)
DATABASE_URL=postgresql://...:6543/postgres # pooled

# AI
ANTHROPIC_API_KEY=...
OPENAI_API_KEY=...

# Sentry
NEXT_PUBLIC_SENTRY_DSN=...
SENTRY_AUTH_TOKEN=...
SENTRY_ORG=...
SENTRY_PROJECT=...
```

```
# Site
NEXT_PUBLIC_SITE_URL=https://gohae.vercel.app
```



Vercel 플랜 가정 — Pro 가정.

- **Pro:** 함수 maxDuration 300s 한도. v1.0은 60s로 설정.
- **Hobby:** 기본 함수 timeout 10s. 60s 설정 시 Vercel이 무시하고 10s에 강제 종료. 메모리 추출이 나 긴 스트림이 중단됨.
- **Hobby 운영 시 조정:** `maxDuration = 10`, `abortSignal.timeout(5_000)`, `generateObject` 호출은 background로 분리 (큐 도입 필요 — v1.x). v1.0 Hobby 운영은 비추.

12. Sentry BE

```
// sentry.server.config.ts
import * as Sentry from '@sentry/nextjs'

const looksLikeUserMessage = (s: string) => /[가-힣\s.,!~?]{30,}/.test(s)

Sentry.init({
  dsn: process.env.NEXT_PUBLIC_SENTRY_DSN,
  tracesSampleRate: 0.1,
  sendDefaultPii: false,
  beforeSend(event) {
    // request body
    if (event.request?.data && typeof event.request.data === 'object') {
      const data = event.request.data as Record<string, unknown>
      if ('messages' in data) data.messages = '[REDACTED]'
      if ('content' in data) data.content = '[REDACTED]'
    }
    // ★ exception value scrub - error 메시지에 인터폴레이트된 사용자 입력 차단
    if (event.exception?.values) {
      for (const ex of event.exception.values) {
        if (ex.value && looksLikeUserMessage(ex.value)) {
          ex.value = '[REDACTED - possible user content]'
        }
      }
    }
    if (event.breadcrumbs) {
      for (const c of event.breadcrumbs) {
        if (c.data && 'body' in c.data) delete (c.data as any).body
      }
    }
    return event
  }
})
```

```
} ,
})
```



규칙 — `throw new Error('실패: ' + userText)` 같은 패턴 금지. 에러 메시지에 사용자 입력 인터플레이트 안 함. ID·코드·상태값만.

배포 직후 T+11:30~12:00 검증 슬롯에서 메시지 원문이 들어가지 않는지 직접 확인.

13. 헬스체크 & 운영

- `/api/health` DB ping (§6.3).
- 로그: 메시지 원문 포함 금지. ID·status·타이밍만.
- 알람 v1.0: Sentry 이슈 이메일.
- 외부 모니터링은 v1.x.

14. 데이터 삭제 (요청 시 수동) — 채널 명시

v1.0은 셀프 UI 없음. 요청 채널: `junkeyong.oh@buildblock.io` (약관·푸터에 명시).

처리 SLA: 14일 내. 처리 방법:

```
delete from auth.users where id = '<USER_UUID>';
-- ON DELETE CASCADE: personas, conversations, messages, memories
```

요청 메일이 도착하면 본인이 직접 처리. v1.x에서 셀프 UI + cron 자동화.

15. 마이그레이션 — SQL 단일 진실 + introspect 단방향



원칙 — SQL이 진실. Drizzle 스키마는 `drizzle-kit introspect`. 양방향 수동 동기화 금지.

흐름

1. `supabase/migrations/0001_init.sql` 작성
2. `supabase/migrations/0002_rls.sql` 작성
3. `supabase db push` (or 대시보드 SQL editor)
4. `npx drizzle-kit introspect` → `lib/db/schema.ts` 자동 갱신
5. 코드에서 `schema.ts` `import`

스키마 변경 (v1.x)

1. 새 SQL 마이그레이션 작성 (`0003_xxx.sql`)

2. local/staging push
 3. `drizzle-kit introspect` 재실행
 4. PR → production push
-

16. 보안 체크리스트

- ☐ 모든 DB 쿼리에 `user_id` 필터 (§4 — RLS는 실질 방어 아님)
 - ☐ persona 의존 흐름은 메시지 INSERT 전에 persona 체크 (§6.1)
 - ☐ 활성 conversation lookup에 `isNull(endedAt)` (§6.1) — 종료된 대화 재사용 차단
 - ☐ zod `InputSchema.role` 은 `['user', 'assistant']` 로 좁힘 — system role 인젝션 차단
 - ☐ 에러 메시지에 사용자 입력 인터폴레이트 금지
 - ☐ RLS policy도 등록 (baseline)
 - ☐ `SUPABASE_SERVICE_ROLE_KEY` 클라이언트 노출 금지
 - ☐ Sentry scrubbing (request + exception.value) 실제 동작 확인
 - ☐ `/api/health` 외 모든 API에 `getCurrentUser()` 게이트
 - ☐ zod로 모든 API 입력 검증
 - ☐ 일일 메시지 cap (KST)
 - ☐ HTTPS (Vercel/Supabase 기본)
 - ☐ OAuth redirect URI 등록 (Google Cloud Console)
 - ☐ **CSRF 의식적 결정**: Route Handler POST는 **Supabase 쿠키 기반 인증 + SameSite=Lax + same-origin** 의존. Server Action은 Next.js 내장 CSRF 토큰 사용. v1.0 별도 토큰 미도입. 외부 origin에서 호출되는 API가 추가되면 그때 명시적 CSRF 토큰 도입 검토.
-

17. 미해결 결정 (v1.x로 이관)

- ☐ 메모리 **supersede** 자동 로직 — 현재 retrieval 시간 가중치 + 시스템 프롬프트로 완화.
- ☐ 위기 LLM 2nd-pass 분류 — 키워드 한계 보완.
- ☐ `summary_status='failed'` **재시도** — cron/수동 트리거.
- ☐ `messages.status='pending' sweep` — 5분 이상 pending 상태를 failed로 전환하는 cron (Vercel disconnect 시 placeholder 영구 pending 방지).
- ☐ **Idempotency-Key** 헤더 — `/api/chat` 중복 호출 방어. 클라이언트가 UUID 헤더로 보내고 BE가 1분 캐시.
- ☐ **Prompt caching** — Claude API의 `cache_control` 로 system 프롬프트(페르소나+메모리) 캐싱 → ~90% 비용 절감. 대화 빈도 늘면 효과 큼.
- ☐ **Retrieval 임계값(0.3)·시간 가중치 상수(60일) 튜닝** — T+11:00 검증 슬롯 + 운영 데이터.
- ☐ 외부 모니터링 (Better Uptime / Vercel monitor).
- ☐ 임베딩 모델 업그레이드 (text-embedding-3-large 등).
- ☐ `auth.users` FK Drizzle relations 표기.

- ☐ 위기 시그널을 AI SDK message annotations로 이관.
- ☐ `messages.role` SQL check를 `('user', 'assistant')` 로 좁히기 — v1.0 코드는 system INSERT 안 하지만 SQL이 허용. 마이그레이션 비용 회피 위해 v1.x.
- ☐ 데이터 삭제 셀프 UI + cron 자동화.
- ☐ `messages.status` 별 FE UI 분기 (placeholder pending / failed 시각 구분).

18. 결정 로그

일자	결정	근거
@2026년 5월 16일	Next.js Route Handlers + Server Actions가 BE 흡수	기획문서 §9.2
@2026년 5월 16일	Vercel Pro 플랜 가정 — maxDuration 60s, abortSignal 45s	Hobby(10s)로는 v1.0 메모리 추출/스트리밍 불가
@2026년 5월 16일	Drizzle ORM + Supabase Postgres pgvector	typescript-first
@2026년 5월 16일	마이그레이션: SQL 단일 진실 + drizzle-kit introspect 단방향	양방향 동기화 무너짐
@2026년 5월 16일	v1.0 Drizzle 경로에서 RLS 미적용. user_id 코드 필터 의무.	슈퍼유저 bypass. 잘못된 안전감 방지
@2026년 5월 16일	활성 conversation은 사용자당 1개 (unique partial index)	단순화
@2026년 5월 16일	위기 신호 채널 = 응답 헤더 <code>x-crisis</code> • DB <code>crisis_flag</code>	컨텐츠/메타시그널 분리
@2026년 5월 16일	응답 헤더 <code>x-conversation-id</code> 항상 세팅	새 conversation ID를 FE에 단일 경로로 전달 (FE v1.2 정합)
@2026년 5월 16일	<code>InputSchema.role</code> 을 <code>['user', 'assistant']</code> 로 좁힘	system role 인젝션 차단
@2026년 5월 16일	persona 체크를 메시지 INSERT 전으로 이동	persona 없는 경로에서 고아 데이터 방지
@2026년 5월 16일	활성 conversation lookup에 <code>isNull(endedAt)</code> 필터	종료된 대화 재사용 차단 (도메인 규칙)
@2026년 5월 16일	외로운 user 메시지 방지: placeholder 패턴 + <code>messages.status</code> enum	데이터 일관성
@2026년 5월 16일	<code>/api/conversation/end</code> : 원자적 UPDATE+RETURNING	race 방지
@2026년 5월 16일	Idempotency: v1.0은 FE disable 의존. v1.x에 Idempotency-Key 헤더	약점 의식적 인정
@2026년 5월 16일	<code>onFinish</code> Vercel disconnect 보장 한계 인정. v1.x에 pending sweep cron	Serverless 비동기 보장 안 됨
@2026년 5월 16일	메모리 모순 완화: retrieval 시간 가중치 + "최신 우선" 시스템 프롬프트	supersede 자동은 v1.x
@2026년 5월 16일	Retrieval 시간 가중치: 30일 → 60일	차별점이 "장기 기억"인데 30일은 짧음
@2026년 5월 16일	Retrieval 쿼리: 마지막 user 메시지 3개 합쳐 임베딩, 10자 미만 스킵	짧은 응답 보호
@2026년 5월 16일	pgvector: hnsw 인덱스 (ivfflat 아님)	v1.0 데이터양 적합
@2026년 5월 16일	Daily cap KST 자정 기준	UX
@2026년 5월 16일	Daily cap 50의 비용 근거: 사용자당 일일 ~\$0.40, 10명 × 30일 ~\$120/월	비용 가시화
@2026년 5월 16일	메모리 추출 max(20) → max(8) + "중요한 3-5개"	인플레이션 방지, retrieval 정밀도

일자	결정	근거
@2026년 5월 16일	Prompt caching v1.x로 이관	v1.0 단순성 우선, 대화 빈도 늘면 효과 큼
@2026년 5월 16일	<code>summary_status</code> enum (<code>pending</code> / <code>completed</code> / <code>failed</code>)	재시도 여지
@2026년 5월 16일	추출 결과는 중복 허용 INSERT	retrieval 시간 가중치 흡수
@2026년 5월 16일	Claude 모델: Sonnet 4.6	품질 우선
@2026년 5월 16일	임베딩: OpenAI text-embedding-3-small	Anthropic 미지원
@2026년 5월 16일	외부 API 재시도 1회만	LLM 품질 retry 무의미
@2026년 5월 16일	메모리 추출은 Server Action 직접 호출 (FE v1.2)	cookie 전달 회피
@2026년 5월 16일	데이터 삭제 셀프 UI v1.x. 요청 채널: <u>junkyong.oh@buildblock.io</u>	SLA 14일 명시
@2026년 5월 16일	외부 모니터링은 v1.x	Sentry 알림 충분
@2026년 5월 16일	위기 안내 어조 + 시간 가중치는 T+11:00~11:30에 본인 검증 필수	차별점/안전 동시 검증
@2026년 5월 16일	CSRF: v1.0 Route Handler는 Supabase 쿠키 인증 + SameSite=Lax + same-origin 의존	의식적 결정, 외부 origin 호출 추가 시 토큰 도입



다음 액션 — FE TechSpec와 정합 완료 (인터페이스 표, `x-conversation-id`, `messages.status`, runtime 규칙). T+1:30~2:30 부트스트랩 슬롯 진입: ① `create-next-app` ② `shadcn init` ③ Supabase 프로젝트·env ④ `0001_init.sql` + `0002_rls.sql` 적용 ⑤ `drizzle-kit introspect` ⑥ Sentry wizard ⑦ 첫 Vercel deploy.